

AI for Economic Research: Dynamic Models, Language, and Agents

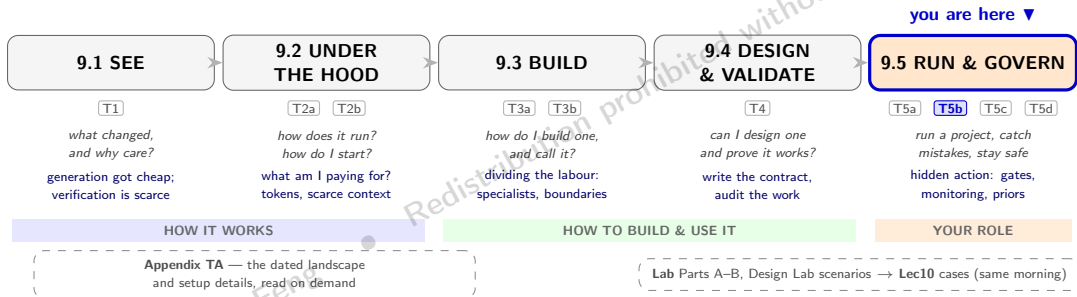
Lecture 9.5b: Management — Organize the Project, Run the RA Team

● Zhigang Feng

2026-09-12 20:06:33

Lecture 9 in One Map: Project Management

This deck answers the next question: how do I run a whole project with this — not just one task?



Route: the paradigm shift (researcher as project manager) → how HA-Ramsey was actually managed → the manager's instruments on disk (instructions files, skills, agents, rules, hooks) → your sub-agent RA team and cross-verification → remote/HPC execution. The economist's question here: how do you write the contract and gate the work?

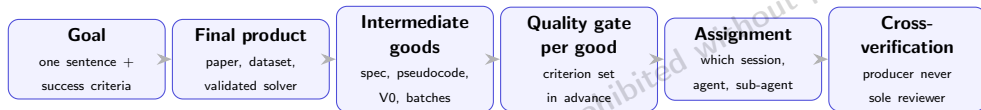
The Paradigm Shift

From solo coder to project manager



The Paradigm Shift: You Are the Project Manager

When execution becomes cheap, the scarce input is management: deciding what gets built, to what standard, by whom.



- Decompose the goal into **intermediate goods** — real artifacts (spec, validated V0, batch CSV, review report), not vague “progress”
- Decide the **quality gate** for each good *before* it is produced (T5a’s acceptance criteria, now for everything, not just code)
- **Assign** each good to the right producer — this session, a sub-agent, a different model tier, or yourself; then **cross-verify**: whoever produced it, someone else checks it

T5a disciplined one artifact; T5b runs the factory. Every object in this deck is one of the manager’s instruments.

A Managed Project in One Slide: HA-Ramsey's Milestones

The HA-Ramsey program is the PM chain instantiated: seven milestones, each a validated, preserved intermediate good.



gate: TeX $\equiv$ code

gate: economist audit

gate: reproduces hand-derived checks

gate: each v1 failure answered

gate: written up, cited

- Every milestone preserved as a citable artifact in the project's Case_Study/ package — *intermediate goods, literally*; no milestone skipped, no gate waived
- The green box is the payoff of management: the discoveries (Story 1 in T1) *surfaced from gated iteration* — they were not the plan

Economics and figures: Lec10 T8. Source: HA-Ramsey case-study package.

Write the Contract First: Who Owns What

Before any code, the HA-Ramsey project wrote down the division of labor — T1's table, now with signatures.

AI owns (throughput)	Economist owns (judgment)
Drafting and revising the LaTeX model note	Economic correctness of every equation
Translating math into vectorized Py-Torch	Which simplifications are acceptable
Penalty/boundary implementation patterns	Catching conceptual bugs in timing and probabilities
Running diagnostics and summarizing sweeps	Validation benchmarks and when a result is credible

“AI supplies throughput; the economist supplies judgment” — stated *before* any code was written. Which goods the AI may produce alone; which gates only you sign. **Collaborating with AI** is working inside it: restate, audit, refine (Lec10 T5).

Quality Gates Catch What No Compiler Catches

Two conceptual bugs the economist caught in AI-drafted equations — the code ran, the losses fell, the results would have been wrong. No compiler catches these:

1. **Wrong timing:** asset transitions written with next-period consumption — but the budget constraint is a *current-period* object
2. **Wrong direction:** transition probabilities indexed by state occupied, not by *direction of movement*

When v1 then failed numerically, each failure became v2 architecture:

v1 failure	v2 structural answer
Naive sampling of a 5D state space	Adaptive sampling loops
Fragile feasible-set boundary learner	Normalized coordinates + robust geometry
Simulated paths escape the valid region	Explicit projection back + penalty term

“Debugging becomes architecture” only if someone with judgment reads the failures — the gate is where the economist earns authorship.

Project Organization

The manager's instruments: files on disk

The PM's Instruments Live on Disk

A manager who re-explains the standards every morning is not managing. Files on disk are how you delegate without re-explaining.

- Conventions live in files, not in your memory — or the agent's
- Validation steps become reproducible lab procedure
- New sessions start from the project's actual state (T2b: initialization reloads it all)
- Coauthors and students can inspect what the workflow is supposed to be

Economist translation: the harness turns tacit RA practice into written lab protocol — and the protocol, not the chat, is what your name ultimately stands behind.

Six Instruments, Six Pain Points

Instrument	Management pain point it solves	Running example
CLAUDE.md	re-explaining the project every session	keep the MEPS weights rule or Aiyagari benchmark visible
Skill	a repeated workflow improvised differently every time	paper-review pipeline with the same quality gates
Agent	review roles need specialization and parallelism	code reviewer vs. domain reviewer
Rule	some instructions must <i>always</i> be on	every filled field carries a source URL
Verifier	output should pass named checks before it counts	rerun script, inspect diagnostics, confirm files exist
Provenance log	source discipline must survive beyond the chat	fellows dataset with field-level URLs and notes

Reason for this slide: these objects are not product trivia. Each one retires a coordination or credibility failure you would otherwise absorb personally.

Minimal Project Tree

```
1 my-research-project/  
2 +-- CLAUDE.md  
3 +-- .claude/  
4 |   +-- skills/  
5 | |   +-- paper-review/  
6 | |       +-- SKILL.md  
7 | |       +-- references/  
8 | |           +-- referee-template.md  
9 |   +-- agents/  
10 | |   +-- code-reviewer.md  
11 | |   +-- domain-reviewer.md  
12 | |   +-- verifier.md  
13 |   +-- rules/  
14 | |   +-- iterative-workflow.md  
15 |   +-- settings.json  
16 +-- scripts/  
17 +-- output/  
18 +-- sources/  
19 +-- quality_reports/
```

This is enough to explain the workflow without burying students in infrastructure. The exact research question can change; the organizational logic stays.

CLAUDE.md: The Standing Brief

CLAUDE.md is the main shared instruction file for the project — the brief a good manager writes once instead of repeating daily.

```
# CLAUDE.md
Project: MEPS uninsurance pilot for lecture 9
Language: Python
Goal: harmonize 2012--2014 files and produce one validated figure
Conventions: no hardcoded paths; save outputs to output/
Checks: use survey weights; save the year map; report what passed
```

Why students should care: it captures the project context you would otherwise re-explain every session.

The fellows project runs the same play with public-web material: the seed CSV, the batch file, and the schema rules are “the public-web analog of CLAUDE.md” (Lec10 T4) — project files holding the rules and state across sessions.

A Corrections File: One Pattern, Any Harness

Every time you correct the agent, write the correction down where the next session will read it. Tag it, date it, keep it short.

```
# CORRECTIONS.md -- accumulated lessons

[LEARN:aiyagari-grid] 2026-09-03
  Too few asset-grid points near the
  borrowing limit. Space them densely
  there; a uniform grid needs 500+.

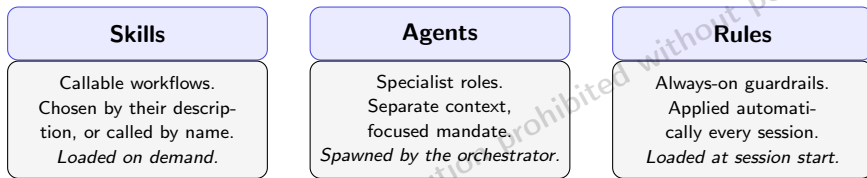
[LEARN:vfi-stop] 2026-09-05
  Never stop on an iteration count.
  Stop on sup-norm < 1e-8 and report
  the final distance.

[LEARN:notation] 2026-09-05
  V value, c_pol consumption policy,
  a_pol savings policy, mu distribution.
```

- **Where it lives:** a file your instructions file points to — CLAUDE.md, GEMINI.md, or AGENTS.md
- **Who writes it:** you, or the agent at your request — then *you* read the diff
- **Why it works:** the next session starts from the file, not from a conversation that was compacted away

The mechanics of what each memory loads and when:
T2b's three memories.

Skills, Agents, and Rules: Three Different Jobs



load on demand → ← spawn on need
Rules load first every session; Skills and Agents enter the loop later.

Loading order matters. Rules are read before the first user message; a skill's body loads when its description matches; an agent starts in a fresh context with its own brief. T3a sorted the same objects by *who supplies the reasoning*; this frame sorts them by *when they load*.

Rules and Hooks: Encoding the Fellows Provenance Discipline

Guidance the agent should follow → a rule. A check that must fire whatever the conversation says → a hook. The Fellows project needs both.

The rule (.claude/rules/provenance.md):

- Every extracted field keeps its own source URL: phd_year -> source_phd, and so on.
- Code gender only when an official source states it explicitly; otherwise leave it missing.
- Record missing as missing, with a note.

Always on: loaded at session start, batch after batch.

The hook (.claude/settings.json):

```
"PreToolUse": [{
  "matcher": "Write",
  "hooks": [{ "type": "command",
    "command":
      "python3 checks/require_source_url.py"}]
}]
```

The script reads the pending write — file path and content — and exits 2: **the row is never written.**

Two ways to get this wrong, both silent (checked on Claude Code 2.1.269). A PostToolUse hook fires *after* the write: it can complain, not prevent. And a matcher such as "Write(data/*.csv)" never fires: matchers name tools, so filter on the file path inside the script. Gemini CLI has comparable BeforeTool hooks (not yet tested here); on any harness, the lab's runner can call the same script.

Permissions and Hooks in settings.json

Two jobs: *permissions* define the blast radius; *hooks* run mechanical checks at tool boundaries.

```
{
  "permissions": {
    "allow": ["Bash(python *)", "Bash(pytest *)", "Read(*)", "Edit(*)"],
    "deny":  ["Bash(rm -rf *)", "Bash(git push --force *)",
              "Read(./env)", "Read(./secrets/**)"]
  },
  "hooks": {
    "PostToolUse": [{"matcher": "Write|Edit",
                     "hooks": [{"type": "command",
                                "command": ".claude/hooks/format-python.sh"}]}],
    "Stop":        [{"hooks": [{"type": "command",
                                "command": "pytest tests/ -q || true"}]}]
  }
}
```

Hook types worth knowing: PreToolUse (block risky commands), PostToolUse (format / lint), Stop (full test suite), SubagentStop (verify report format), PreCompact (save progress).

Three Lifetimes: Per-Event, Per-Session, Forever

The harness is the umbrella name for what makes a project survive across sessions.

Layer	Lifetime	Purpose
Hooks (settings.json)	Per-event	Mechanical enforcement: format after edits, run tests on stop, block dangerous tools
Project memory / rules	Per-session	Bootstrap context. Project instructions and path-scoped guardrails
Auto memory + progress files	Cross-session	Durable learnings and explicit handoff notes you can inspect and edit

Why this matters. Choosing the right layer is half the design: if the rule must fire every time, a hook is safer than a natural-language reminder; if it must survive a fresh session, it belongs in memory or rules, not a transient prompt.

Rule of thumb: natural-language guidance → rules and skills. Mechanical enforcement → hooks.

Your RA Team

Sub-agents, assignment, and cross-verification

Sub-Agents: Your Parallel RA Team

The files you wrote in T3a, managed as a team: each sub-agent is a separate instance with its own context window (T3b).

- Multiple sub-agents run **in parallel** — like Python's multiprocessing
- Each receives an agent file as its system prompt and returns a structured report
- The specialist roles are the RA team you met in the project tree:
 - `code-reviewer.md`: file hygiene, implementation quality, reproducibility
 - `domain-reviewer.md`: benchmark logic, identification, economic consistency
 - `verifier.md`: rerun the script, inspect diagnostics, confirm outputs exist

Aiyagari example. After writing `aiyagari_v1.py`, the orchestrator spawns

```
code-reviewer || domain-reviewer
```

in parallel: less wall-clock time (T3b's ledger: 5.6 minutes against 13.5), more tokens, and the same framing (T5c). *Why separate roles? One prompt asking about everything does a mediocre job at each — one RA checks style, another checks economics, another reruns the table.*

Assignment and Cross-Verification

The manager's two questions for every intermediate good: who produces it, and who checks it?

Good fit for sub-agents:

- Code review (parallel: code, domain, verification)
- Lookups and web research that return short reports
- Independent unit-test generation per module

Bad fit:

- Tightly coupled work (each step depends on the previous)
- Anything you must steer interactively mid-task
- Sub-agents spawning sub-agents (depth causes drift)

Cross-verification: the producer is never the only reviewer. Blast-radius rules of thumb: plan mode before granting execution on novel pipelines; each sub-agent's checklist *spec'd*, not free-form; checkpoint validation between phases (the homotopy gates from T3).

Foreshadow for T5c: parallel reviewers are parallel executionally, not epistemically — they read the same project files you wrote.

Orchestration at Research Scale

The HA-Ramsey lesson list ends with a management skill: to benefit the most, you need four things.

1. **A good seed** — T5a's criteria: works, documented, modular, points somewhere
2. **A clear vision** — the final product and the milestone ladder to it
3. **Good judgment** — the quality gates only an economist can sign
4. **Orchestration** — run *different sessions and agents on different layers*, then integrate

What orchestration looked like in practice:

- Parallel sessions for formalization (TeX), implementation (PyTorch), auditing (code-vs-doc), sweeps, and write-up — all feeding one preserved package
- 32+ configuration sweeps run and summarized by agents; the economist read summaries and decided at the milestones — the job is *integration*: the PM chain on a real paper

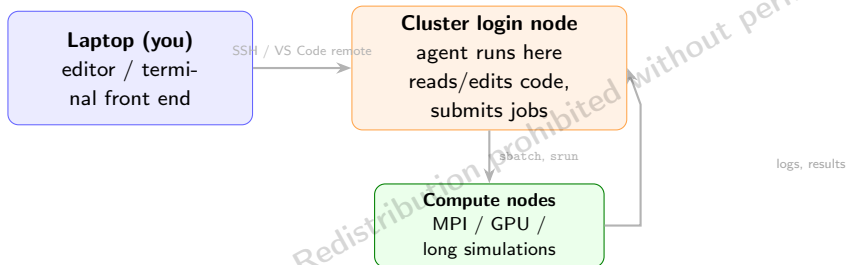
Source: HA-Ramsey case-study package (Lec10 T8).

Remote and HPC

When the laptop is no longer the right host

Remote/HPC: Move the Agent to the Compute

For heavy workloads the agent should run where the data, modules, and GPUs already are.



- With a local agent, cluster files must be copied back and forth — easy for toy code, fragile for research
- With a remote agent, project files stay in the remote worktree; your laptop is only the front end

Rule: if the data, modules, GPUs, and job logs live on the cluster, install the agent on the cluster.

Login Node vs. Compute Node: The Hard Boundary

What the agent can do on a login node:

- Read/edit code, specs, context files, and job scripts
- Run tiny smoke tests: import checks, unit tests, `-dry-run`, small-grid V0
- Submit jobs, monitor logs, parse errors, and revise scripts

What must go through the scheduler:

- MPI jobs, GPU training, large simulations, long calibrations
- Anything that consumes many cores, much memory, or more than a few minutes

Research rule: the agent may orchestrate HPC work; it should not turn the login node into a workstation.

Bridge: Management Assumes Trust — Verify It

You now delegate through files and a sub-agent team. Delegation works only if you can audit what was actually done — and know when to distrust the polish.

- T5b gave you the management system: goods, gates, assignment, cross-verification, and the instruments that encode them
- The final question: *“what can go wrong, and what is my job?”*
- T5c answers with three moves: **trace** what the agent did, catalog the **failure modes** the harness cannot prevent, and confront the **Context Dilemma** — the same context that improves the answer also pulls it toward you

Arc: 9.1 See → 9.2 Under the Hood → 9.3 Build → 9.4 Design & Validate → **9.5 Run & Govern.**